

APEC 8221 - Assignment 2: Automation and API Data Acquisition

Submission File

[Runcheng Xu]

2025-09-27

Table of contents

1	Build Your API Function	2
1.1	API Setup and Testing	2
1.2	Write Your Data Acquisition Function	5
2	Automate Data Collection	7
2.1	Define Your State List	7
2.2	Implement Rate Limiting	7
2.3	Compare Iteration Methods	8
2.3.1	Method 1: for Loop Approach	8
2.3.2	Method 2: lapply Approach	10
2.3.3	Method 3: purrr::map Approach	10
2.3.4	Method 4: Manual Approach (Demonstration)	12
3	Data Analysis and Visualization	12
3.1	Regional Unemployment Patterns	13
3.1.1	Basic Summary Statistics	13
3.2	Time Series Visualization	14
3.3	State Comparison Analysis	15
4	Extension and Critical Thinking	17
4.1	API Pagination Challenge	17
4.1.1	Step 1: Explore API Metadata	17
4.1.2	Step 2: Build Complete Data Function	18
4.1.3	Step 3: Pagination Analysis	20
4.2	Alternative API Practice	20
4.2.1	API Documentation Research	20

4.2.2	Custom API Function	21
4.2.3	Data Collection and Analysis	22
4.2.4	Visualization	23
4.2.5	API Comparison and Reflection	24
4.3	API Landscape Survey	24
4.3.1	Quick API Exploration	24
4.3.2	API Structure Analysis	25

GitHub Repository URL: <https://github.com/Runcheng112/apec8221-xu001296.git>

1 Build Your API Function

```
# Load required packages
library(tidyverse)
library(httr2)
library(jsonlite)

# Secure API key storage - load from .Renviron file
fred_api_key <- Sys.getenv("FRED_API_KEY")
```

1.1 API Setup and Testing

```
# Test basic API connectivity
# Your code here to test the FRED API connection
base_url <- "https://api.stlouisfed.org/fred/series/observations"
test_url <- paste0(base_url, "?", "series_id=UNRATE",
"&api_key=", fred_api_key, "&file_type=json")
# Make the request
response <- request(test_url) %>% req_perform()
# Check if it worked
resp_status(response) # Should be 200
```

```
[1] 200
```


[illegible]

[illegible]

1.2 Write Your Data Acquisition Function

```
library(httr2)
library(dplyr)

get_fred_unemployment <- function(state_code, api_key) {
  # Input validation
```

```

if (!grepl("^[A-Z]{2}$", state_code)) {
  stop("state_code must be two capital letters, e.g., 'CA' or 'NY'")
}

series_id <- paste0(state_code, "URN")
base_url <- "https://api.stlouisfed.org/fred/series/observations"
request_url <- paste0(
  base_url,
  "?series_id=", series_id,
  "&api_key=", api_key,
  "&file_type=json"
)

response <- request(request_url) %>% req_perform()

if (resp_status(response) != 200) {
  stop(paste("API request failed. Status:", resp_status(response)))
}

raw_data <- response %>% resp_body_json()
if (is.null(raw_data$observations)) {
  stop("No data returned. Check state_code.")
}

result <- raw_data$observations %>%
  purrr::map_df(~{
    data.frame(
      date = as.Date(.x$date),
      unemployment_rate = as.numeric(.x$value),
      state = state_code,
      stringsAsFactors = FALSE
    )
  })

return(result)
}

# Test
test_result <- get_fred_unemployment("CA", fred_api_key)
head(test_result)

```

```

date unemployment_rate state

```

1	1976-01-01	10.5	CA
2	1976-02-01	10.1	CA
3	1976-03-01	9.4	CA
4	1976-04-01	8.8	CA
5	1976-05-01	7.8	CA
6	1976-06-01	8.9	CA

2 Automate Data Collection

2.1 Define Your State List

```
# Select 8-10 states from different regions
target_states <- c("CA", "TX", "NY", "FL", "OH", "WA", "CO", "GA")

# Print your selected states
```

2.2 Implement Rate Limiting

```
library(httr2)
library(dplyr)

get_fred_unemployment <- function(state_code, api_key) {
  # Input validation
  if (!grepl("^[A-Z]{2}$", state_code)) {
    stop("state_code must be two capital letters, e.g., 'CA' or 'NY'")
  }

  cat("Downloading data for", state_code, "...\\n")

  Sys.sleep(1) # wait 1 second before API call

  series_id <- paste0(state_code, "URN")
  base_url <- "https://api.stlouisfed.org/fred/series/observations"
  request_url <- paste0(
    base_url,
    "?series_id=", series_id,
    "&api_key=", api_key,
    "&file_type=json"
  )
}
```

```

)

response <- request(request_url) %>% req_perform()

# Check response status
if (resp_status(response) != 200) {
  stop(paste("API request failed. Status:", resp_status(response)))
}

raw_data <- response %>% resp_body_json()
if (is.null(raw_data$observations)) {
  stop("No data returned. Check state_code.")
}

result <- raw_data$observations %>%
  purrr::map_df(~{
    data.frame(
      date = as.Date(.x$date),
      unemployment_rate = as.numeric(.x$value),
      state = state_code,
      stringsAsFactors = FALSE
    )
  })

return(result)
}

```

2.3 Compare Iteration Methods

2.3.1 Method 1: for Loop Approach

```

# Initialize empty data frame
unemployment_data_loop <- data.frame()

# Loop through states
for (state in target_states) {
  cat("Downloading data for", state, "...\\n")

  # Get data for this state
  state_data <- get_fred_unemployment(state, fred_api_key)
}

```



```

# Add it to unemployment_data_loop
unemployment_data_loop <- rbind(unemployment_data_loop, state_data)

# Pause to respect API rate limits
Sys.sleep(1)
}

```

```

Downloading data for CA ...
Downloading data for CA ...
Downloading data for TX ...
Downloading data for TX ...
Downloading data for NY ...
Downloading data for NY ...
Downloading data for FL ...
Downloading data for FL ...
Downloading data for OH ...
Downloading data for OH ...
Downloading data for WA ...
Downloading data for WA ...
Downloading data for CO ...
Downloading data for CO ...
Downloading data for GA ...
Downloading data for GA ...

```

```

# Check results
head(unemployment_data_loop)

```

	date	unemployment_rate	state
1	1976-01-01	10.5	CA
2	1976-02-01	10.1	CA
3	1976-03-01	9.4	CA
4	1976-04-01	8.8	CA
5	1976-05-01	7.8	CA
6	1976-06-01	8.9	CA

```

# Check results

```

2.3.2 Method 2: lapply Approach

```
unemployment_list <- lapply(target_states, function(state) {  
  cat("Downloading data for", state, "...\\n")  
  Sys.sleep(1) # pause to respect API limits  
  get_fred_unemployment(state, fred_api_key)  
})
```

```
Downloading data for CA ...  
Downloading data for CA ...  
Downloading data for TX ...  
Downloading data for TX ...  
Downloading data for NY ...  
Downloading data for NY ...  
Downloading data for FL ...  
Downloading data for FL ...  
Downloading data for OH ...  
Downloading data for OH ...  
Downloading data for WA ...  
Downloading data for WA ...  
Downloading data for CO ...  
Downloading data for CO ...  
Downloading data for GA ...  
Downloading data for GA ...
```

```
unemployment_data_apply <- do.call(rbind, unemployment_list)  
  
head(unemployment_data_apply)
```

	date	unemployment_rate	state
1	1976-01-01	10.5	CA
2	1976-02-01	10.1	CA
3	1976-03-01	9.4	CA
4	1976-04-01	8.8	CA
5	1976-05-01	7.8	CA
6	1976-06-01	8.9	CA

2.3.3 Method 3: purrr::map Approach

```

library(purrr)
library(dplyr)

# Use purrr::map to get data for all states
unemployment_data_purrr <- target_states %>%
  map(function(state) {
    cat("Downloading data for", state, "...\\n")
    Sys.sleep(1) # pause to respect API limits
    get_fred_unemployment(state, fred_api_key)
  }) %>%
  bind_rows()

```

```

Downloading data for CA ...
Downloading data for CA ...
Downloading data for TX ...
Downloading data for TX ...
Downloading data for NY ...
Downloading data for NY ...
Downloading data for FL ...
Downloading data for FL ...
Downloading data for OH ...
Downloading data for OH ...
Downloading data for WA ...
Downloading data for WA ...
Downloading data for CO ...
Downloading data for CO ...
Downloading data for GA ...
Downloading data for GA ...

```

```

# Check results
head(unemployment_data_purrr)

```

	date	unemployment_rate	state
1	1976-01-01	10.5	CA
2	1976-02-01	10.1	CA
3	1976-03-01	9.4	CA
4	1976-04-01	8.8	CA
5	1976-05-01	7.8	CA
6	1976-06-01	8.9	CA

2.3.4 Method 4: Manual Approach (Demonstration)

```
# Get data for each state one by one
ca_data <- get_fred_unemployment("CA", fred_api_key)
tx_data <- get_fred_unemployment("TX", fred_api_key)
ny_data <- get_fred_unemployment("NY", fred_api_key)
fl_data <- get_fred_unemployment("FL", fred_api_key)
oh_data <- get_fred_unemployment("OH", fred_api_key)

unemployment_data_manual <- rbind(ca_data, tx_data, ny_data, fl_data, oh_data)

# Check results
head(unemployment_data_manual)
```

Iteration Method Comparison: The for loop is simple and explicit, but less concise. `lapply` is more efficient and compact, while `purrr::map` is the most readable and integrates well with tidyverse workflows. Automation is valuable because it reduces errors and saves time when working with multiple states.

Scaling Analysis:

For all 50 states, the same code can be used by extending the state list. The main challenges are API rate limits and longer run times. `for` loops may become slow, while `lapply` and `purrr::map` handle larger scale more efficiently.

3 Data Analysis and Visualization

```
# Use the data from your preferred method above
unemployment_data <- unemployment_data_loop # or unemployment_data_apply or ...

# Check data structure
glimpse(unemployment_data)
```

```
Rows: 4,768
Columns: 3
```

```
$ date          <date> 1976-01-01, 1976-02-01, 1976-03-01, 1976-04-01, 197~
$ unemployment_rate <dbl> 10.5, 10.1, 9.4, 8.8, 7.8, 8.9, 9.5, 9.1, 8.9, 8.9, ~
$ state         <chr> "CA", "CA", "CA", "CA", "CA", "CA", "CA", "CA", "CA"~
```

3.1 Regional Unemployment Patterns

3.1.1 Basic Summary Statistics

```
library(dplyr)

state_averages <- unemployment_data %>%
  filter(date >= "2019-01-01" & date <= "2024-12-31") %>%
  group_by(state) %>%
  summarise(avg_rate = mean(unemployment_rate, na.rm = TRUE)) %>%
  arrange(desc(avg_rate))

highest_peak <- unemployment_data %>%
  filter(date >= "2019-01-01" & date <= "2024-12-31") %>%
  group_by(state) %>%
  summarise(max_rate = max(unemployment_rate, na.rm = TRUE)) %>%
  arrange(desc(max_rate)) %>%
  slice(1)

volatility_analysis <- unemployment_data %>%
  filter(date >= "2019-01-01" & date <= "2024-12-31") %>%
  group_by(state) %>%
  summarise(sd_rate = sd(unemployment_rate, na.rm = TRUE)) %>%
  arrange(desc(sd_rate)) %>%
  slice(1)

print(state_averages)
```

```
# A tibble: 8 x 2
  state avg_rate
  <chr>   <dbl>
1 CA      5.98
2 NY      5.59
3 WA      5.15
4 OH      4.92
5 TX      4.82
```

6	CO	4.28
7	FL	4.23
8	GA	4.02

```
print(highest_peak)
```

```
# A tibble: 1 x 2
  state max_rate
  <chr>   <dbl>
1 NY      16.7
```

```
print(volatility_analysis)
```

```
# A tibble: 1 x 2
  state sd_rate
  <chr>   <dbl>
1 NY      2.82
```

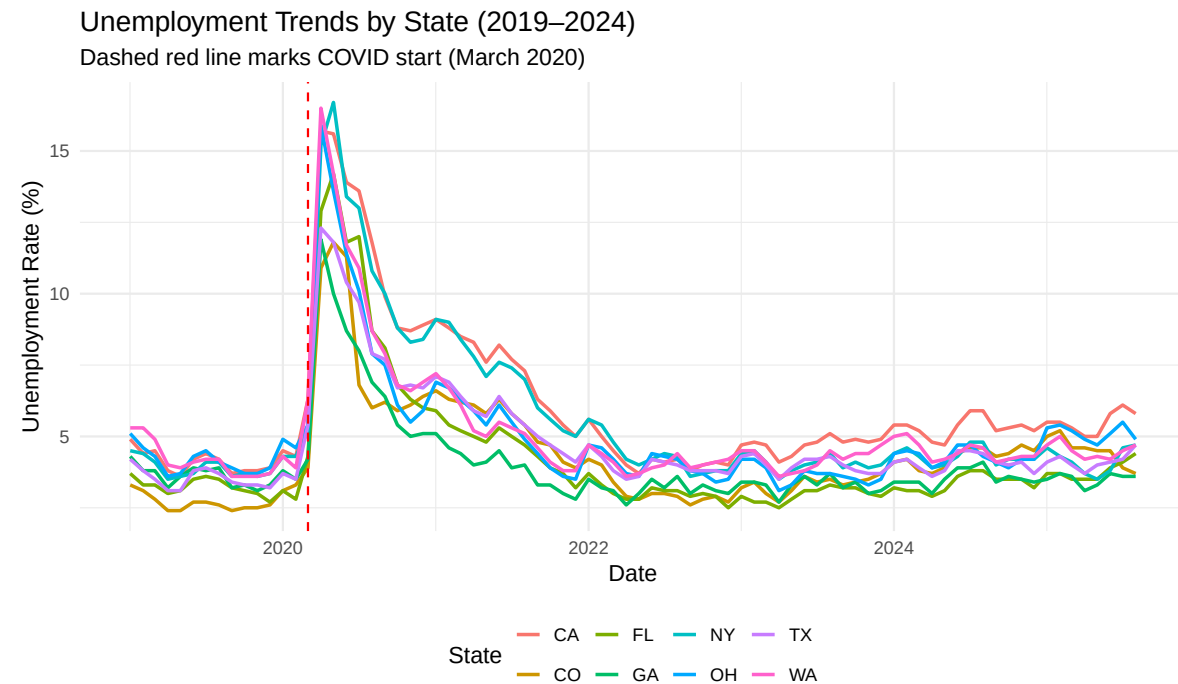
Key Findings:

CA has the highest average unemployment rate (5.98%), while **GA** has the lowest (4.02%)
NY shows both the highest peak (16.7%) and the greatest volatility (SD 2.82).

3.2 Time Series Visualization

```
# Create time series plot
unemployment_plot <- unemployment_data %>%
  filter(date >= as.Date("2019-01-01")) %>%
  ggplot(aes(x = date, y = unemployment_rate, color = state)) +
  geom_line(size = 1) +
  geom_vline(xintercept = as.Date("2020-03-01"),
            linetype = "dashed", color = "red") +
  labs(title = "Unemployment Trends by State (2019-2024)",
       subtitle = "Dashed red line marks COVID start (March 2020)",
       x = "Date",
       y = "Unemployment Rate (%)",
       color = "State") +
  theme_minimal(base_size = 14) +
  theme(legend.position = "bottom")
```

```
print(unemployment_plot)
```



Analysis: Unemployment rates spiked sharply across all states in March 2020, reflecting the immediate impact of COVID-19. Rates then gradually declined, though the speed of recovery varied by state. By 2022–2024, unemployment stabilized at lower levels, but small differences across states remained.

3.3 State Comparison Analysis

```
library(dplyr)
library(ggplot2)

unemployment_comparison <- unemployment_data %>%
  mutate(period = case_when(
    date >= as.Date("2019-01-01") & date <= as.Date("2020-02-29") ~ "Pre-COVID",
    date >= as.Date("2023-01-01") & date <= as.Date("2024-12-31") ~ "Post-COVID",
    TRUE ~ NA_character_
  )) %>%
  filter(!is.na(period)) %>%
```

```

group_by(state, period) %>%
  summarise(avg_rate = mean(unemployment_rate, na.rm = TRUE), .groups = "drop")

unemployment_comparison <- unemployment_comparison %>%
  group_by(state) %>%
  arrange(state, desc(period)) %>%
  ungroup() %>%
  mutate(state = reorder(state, avg_rate))

comparison_plot <- ggplot(unemployment_comparison, aes(x = state, y = avg_rate, fill = period)) +
  geom_col(position = "dodge") +
  labs(title = "Average Unemployment Rate by State (Pre vs Post COVID)",
       x = "State",
       y = "Average Unemployment Rate (%)",
       fill = "Period") +
  theme_minimal(base_size = 14) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

print(comparison_plot)

```



Analysis:

The bar chart shows that unemployment rates generally increased in the Post-COVID period (2023–2024) compared to the Pre-COVID period (2019–early 2020). States like California (CA) and Colorado (CO) saw the largest increases, while Florida (FL) remained relatively stable. This suggests that COVID had long-term effects on unemployment, with stronger impacts in some states than others.

4 Extension and Critical Thinking

4.1 API Pagination Challenge

4.1.1 Step 1: Explore API Metadata

```
# Explore pagination with a limited request
response <- request("https://api.stlouisfed.org/fred/series/observations") %>%
  req_url_query(
    series_id = "CAUR", # California unemployment
    api_key = fred_api_key,
    file_type = "json",
    limit = 100 # Small limit to see pagination
  ) %>%
  req_perform()

full_data <- response %>% resp_body_json()

# Examine metadata structure
str(full_data, max.level = 1)
```

List of 13

```
$ realtime_start : chr "2025-09-22"
$ realtime_end   : chr "2025-09-22"
$ observation_start: chr "1600-01-01"
$ observation_end : chr "9999-12-31"
$ units          : chr "lin"
$ output_type    : int 1
$ file_type      : chr "json"
$ order_by       : chr "observation_date"
$ sort_order     : chr "asc"
```

```

$ count          : int 596
$ offset         : int 0
$ limit          : int 100
$ observations    :List of 100

```

Metadata Analysis: Metadata Analysis

The FRED API provides pagination information through the parameters **limit** (how many observations per call) and **offset** (starting index for the data). The metaata also shows **count**, which indicates the **total number of available observations** (here it is 596). In the first call, with `limit = 100`, the API returned **100 observations** out of the total 596.

4.1.2 Step 2: Build Complete Data Function

```

get_fred_unemployment_complete <- function(state_code, api_key, page_size = 100) {

  # Step 1: Get metadata to find total number of observations
  meta_response <- request("https://api.stlouisfed.org/fred/series/observations") %>%
    req_url_query(
      series_id = state_code,
      api_key = api_key,
      file_type = "json",
      limit = 1 # only need 1 record to get metadata
    ) %>%
    req_perform()

  meta_data <- meta_response %>% resp_body_json()
  total_obs <- as.numeric(meta_data$count)

  message("Total observations available: ", total_obs)

  # Step 2: Calculate number of pages needed
  num_pages <- ceiling(total_obs / page_size)
  message("Fetching data in ", num_pages, " pages...")

  # Step 3: Loop through pages and collect data
  all_data <- list()

  for (i in 0:(num_pages - 1)) {
    offset_val <- i * page_size

```

```

message("Getting page ", i + 1, " of ", num_pages, " (offset = ", offset_val, ")")

page_response <- request("https://api.stlouisfed.org/fred/series/observations") %>%
  req_url_query(
    series_id = state_code,
    api_key = api_key,
    file_type = "json",
    limit = page_size,
    offset = offset_val
  ) %>%
  req_perform()

page_data <- page_response %>% resp_body_json()

# Extract observations
all_data[[i + 1]] <- tibble::tibble(
  date = as.Date(sapply(page_data$observations, function(x) x$date)),
  value = as.numeric(sapply(page_data$observations, function(x) x$value))
)
}

# Step 4: Combine and return complete dataset
complete_data <- dplyr::bind_rows(all_data)
return(complete_data)
}

ca_data <- get_fred_unemployment_complete("CAUR", fred_api_key, page_size = 100)
head(ca_data)

```

```

# A tibble: 6 x 2
  date      value
<date>    <dbl>
1 1976-01-01  9.2
2 1976-02-01  9.2
3 1976-03-01  9.1
4 1976-04-01   9
5 1976-05-01  8.9
6 1976-06-01  8.9

```

4.1.3 Step 3: Pagination Analysis

```
# Compare complete vs. recent data
# Your analysis code here

# Answer the required questions:
# 1. How many total observations for California?
# 2. How many API calls needed with page_size = 500?
# 3. Trade-offs between page sizes?
# 4. When to use complete vs. recent data?
```

Pagination Insights:

1. For California unemployment data, there are **596 total observations** available, as shown in the API metadata
2. If a **page size of 500** is used, the dataset would require **2 API calls**: the first call would retrieve 500 records, and the second call would fetch the remaining 96 records.
3. The trade-offs between page sizes involve efficiency versus reliability. **Larger page sizes** reduce the number of API calls, improving performance, but they may risk timeouts or memory issues if the dataset is very large. **Smaller page sizes**, on the other hand, are more reliable and easier to handle, but they require more calls, which can slow down processing and increase server load.
4. **Complete historical data** is useful for long-term trend analysis, policy evaluation, and studying structural changes in the labor market. **Recent data only** is more appropriate for short-term forecasting, real-time monitoring, or analyzing current economic conditions without the overhead of handling the entire historical dataset.

4.2 Alternative API Practice

Selected API: *Option B (BLS*

4.2.1 API Documentation Research

The BLS API requires sending series IDs and an API key in the request body, and the response is returned in a nested JSON format with results grouped under **"Results"** and **"series"**. This structure is different from FRED, which uses query parameters in the URL and provides metadata like observation counts and pagination controls more explicitly.

4.2.2 Custom API Function

```
bls_api_key <- "74c72444bdac46fd9977d3af407c9d7e"
library(httr)
library(jsonlite)
library(dplyr)

bls_api_key <- "74c72444bdac46fd9977d3af407c9d7e"

# Custom function for BLS API
get_bls_data <- function(series_id, startyear = 2019, endyear = 2024, api_key = bls_api_key)
  url <- "https://api.bls.gov/publicAPI/v2/timeseries/data/"

  response <- POST(
    url,
    body = list(
      seriesid = list(series_id),
      startyear = startyear,
      endyear = endyear,
      registrationkey = api_key
    ),
    encode = "json"
  )

  data <- content(response, as = "parsed", type = "application/json")

  df <- data.frame(
    year = sapply(data$Results$series[[1]]$data, function(x) x$year),
    period = sapply(data$Results$series[[1]]$data, function(x) x$period),
    value = sapply(data$Results$series[[1]]$data, function(x) x$value)
  )

  df <- df %>%
    filter(grepl("M", period)) %>% #
    mutate(
      date = as.Date(paste0(year, "-", substring(period, 2), "-01")),
      value = as.numeric(value)
    ) %>%
    arrange(date)

  return(df)
}
```

```
# Test your function
test_result <- get_bls_data("CES0500000003", startyear = 2019, endyear = 2024)
head(test_result)
```

	year	period	value	date
1	2019	M01	27.59	2019-01-01
2	2019	M02	27.69	2019-02-01
3	2019	M03	27.77	2019-03-01
4	2019	M04	27.79	2019-04-01
5	2019	M05	27.88	2019-05-01
6	2019	M06	27.96	2019-06-01

4.2.3 Data Collection and Analysis

```
library(dplyr)
library(purrr)

target_entities <- c("CES0500000003", # Total private hourly earnings
                    "CES0600000003", # Manufacturing hourly earnings
                    "CES0800000003") # Construction hourly earnings

api_data <- map_df(target_entities, function(id) {
  df <- get_bls_data(id, startyear = 2019, endyear = 2024)
  df$series_id <- id
  return(df)
})

summary_stats <- api_data %>%
  group_by(series_id) %>%
  summarise(
    avg_value = mean(value, na.rm = TRUE),
    max_value = max(value, na.rm = TRUE),
    min_value = min(value, na.rm = TRUE)
  )

head(summary_stats)
```

```
# A tibble: 3 x 4
  series_id      avg_value max_value min_value
```

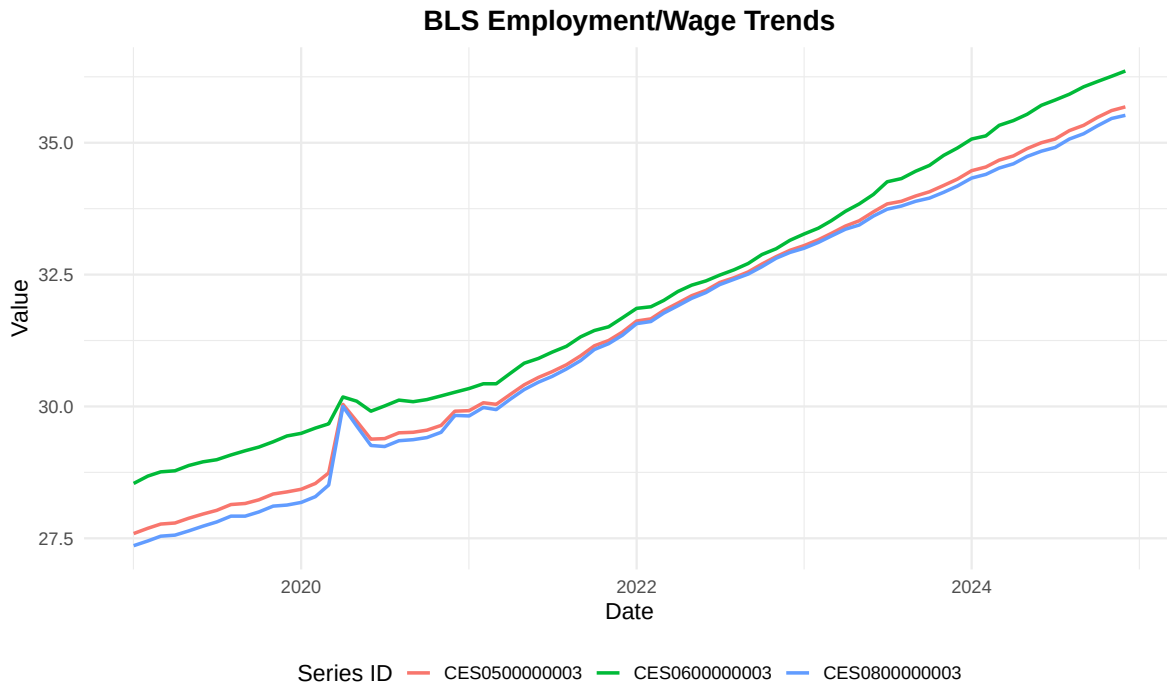
	<chr>	<dbl>	<dbl>	<dbl>
1	CES05000000003	31.5	35.7	27.6
2	CES06000000003	32.0	36.4	28.5
3	CES08000000003	31.4	35.5	27.4

4.2.4 Visualization

```
library(ggplot2)

api_plot <- ggplot(api_data, aes(x = date, y = value, color = series_id)) +
  geom_line(size = 1) +
  labs(
    title = "BLS Employment/Wage Trends",
    x = "Date",
    y = "Value",
    color = "Series ID"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    legend.position = "bottom"
  )

print(api_plot)
```



4.2.5 API Comparison and Reflection

Working with the BLS API was more challenging than FRED because the data comes in a nested JSON structure and requires handling request bodies rather than just URL queries. This taught me that different APIs vary in format and complexity, so flexibility and careful reading of documentation are key when adapting code.

4.3 API Landscape Survey

4.3.1 Quick API Exploration

```
# Explore three different API structures without writing full functions

# API 1: NASA Astronomy Picture of the Day (nested JSON, government API)
nasa_response <- request("https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY") %>%
  req_perform()
nasa_data <- nasa_response %>% resp_body_json()
str(nasa_data)
```


List of 8

```
$ copyright      : chr "Pascal Fouquet"
$ date           : chr "2025-09-27"
$ explanation     : chr "On the morning of September 24 a rocket crosses the bright solar di
$ hdurl           : chr "https://apod.nasa.gov/apod/image/2509/IMAP-IG2-001.JPG"
$ media_type      : chr "image"
$ service_version: chr "v1"
$ title           : chr "A Rocket in the Sun"
$ url             : chr "https://apod.nasa.gov/apod/image/2509/IMAP-IG2-001_1024.JPG"
```

```
# API 2: JSONPlaceholder (flat JSON, RESTful design)
posts_response <- request("https://jsonplaceholder.typicode.com/posts/1") %>%
  req_perform()
posts_data <- posts_response %>% resp_body_json()
str(posts_data)
```

List of 4

```
$ userId: int 1
$ id     : int 1
$ title  : chr "sunt aut facere repellat provident occaecati excepturi optio reprehenderit"
$ body   : chr "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehendit"
```

```
# API 3: Cat Facts (simple response, no authentication)
facts_response <- request("https://catfact.ninja/fact") %>%
  req_perform()
facts_data <- facts_response %>% resp_body_json()
str(facts_data)
```

List of 2

```
$ fact : chr "Cats control the outer ear using 32 muscles; humans use 6"
$ length: int 57
```

4.3.2 API Structure Analysis

1. The response structures differ from FRED because FRED returns tabular time-series data, while NASA uses nested JSON with text and URLs, JSONPlaceholder gives flat JSON, and Cat Facts provides a very simple one-field JSON.
2. NASA requires an API key (though a demo key works), while JSONPlaceholder and Cat Facts require no authentication

3. Government APIs (like NASA, FRED, BLS) are more structured and secure, commercial APIs often include pagination and strict key use, while fun APIs are lightweight and mainly for practice.
4. Across APIs, common patterns include JSON responses, frequent use of keys for authentication, and parameters for pagination or limits, but structures range from highly nested to very flat.